Cahier des Charges

Autonomous Mobile Robot Optimus - EmbodiedAI

Prepared by: Wajdi Dridi - Chair of IEEE RAS SEN Student Branch Joint Chapter

October 23, 2025

Abstract

This document specifies functional requirements, hardware and software architecture, interfaces, testing and acceptance criteria for a autonomous robotic system:

1. **Real-Time Control Module (STM32F407)** — handles low-level actuation, battery management, and safety.
2. **Autonomous Intelligence Module (Raspberry Pi 4)** — manages perception, decision-making, AI computation, and communication with PC-based supervision via MQTT.

Objective: create a robot capable of operating autonomously for inspection, data logging, and environmental awareness.

Contents

1 Project Overview **3**
1.1 Objectives . . . 3

2 Scope and Constraints **3**

3 Functional Requirements **3**
3.1 Real-Time Control Module (STM32F407) . . . 3
3.2 Autonomous Intelligence Module (Raspberry Pi 4) . . . 3

4 System Architecture **4**
4.1 Global Overview . . . 4
4.2 Communication Interfaces . . . 4

5 Hardware Components and Specifications **5**

6 Software Architecture and Data Acquisition **5**
6.1 Embedded Firmware (STM32F407) . . . 5
6.2 Autonomous Intelligence (Raspberry Pi 4) . . . 5
6.3 Data Acquisition Using Mosquitto and Node-RED . . . 5

1 Project Overview

1.1 Objectives

- Develop a hybrid mobile robot, both remote-controlled and autonomous.
- Integrate STM32F407 for real-time control and FreeRTOS multitasking.
- Utilize Raspberry Pi 4 for AI-based decision-making and risk prediction.
- Design MQTT network via Mosquitto broker for real-time telemetry acquisition.
- Implement Node-RED dashboard for data visualization and command control.

2 Scope and Constraints

- **Surface:** Flat indoor terrain, small thresholds $<1.5\,\text{cm}$.
- **Power:** LiPo 4S battery, BMS integration.
- **Control:** Tasks scheduled under FreeRTOS (STM32).
- **Communication:** MQTT over WiFi using ESP32 or direct WLAN.
- **Footprint:** Max 600 mm $\times$ 600 mm frame.

3 Functional Requirements

3.1 Real-Time Control Module (STM32F407)

FRR1: PWM motor control (4WD system).

FRR2: Ultrasonic and IMU-based distance and motion detection.

FRR3: FreeRTOS task separation for motion, telemetry, safety, and communication.

FRR4: Battery monitoring and analog signal conditioning.

FRR5: UART bridge communication with Raspberry Pi and ESP32 (WiFi).

FRR6: GPIO-triggered emergency stop for safety.

3.2 Autonomous Intelligence Module (Raspberry Pi 4)

FRA1: Camera-based visual navigation with OpenCV.

FRA2: DHT22 environmental data logging.

FRA3: AI risk prediction using TensorFlow Lite (MobileNet).

FRA4: Sensor diagnostics monitoring data integrity and failures.

FRA5: Emergency response behaviors (stop, reroute, alert).

FRA6: MQTT client publishing telemetry and subscribing to commands.

FRA7: Node-RED data acquisition and control dashboard.

4 System Architecture

4.1 Global Overview

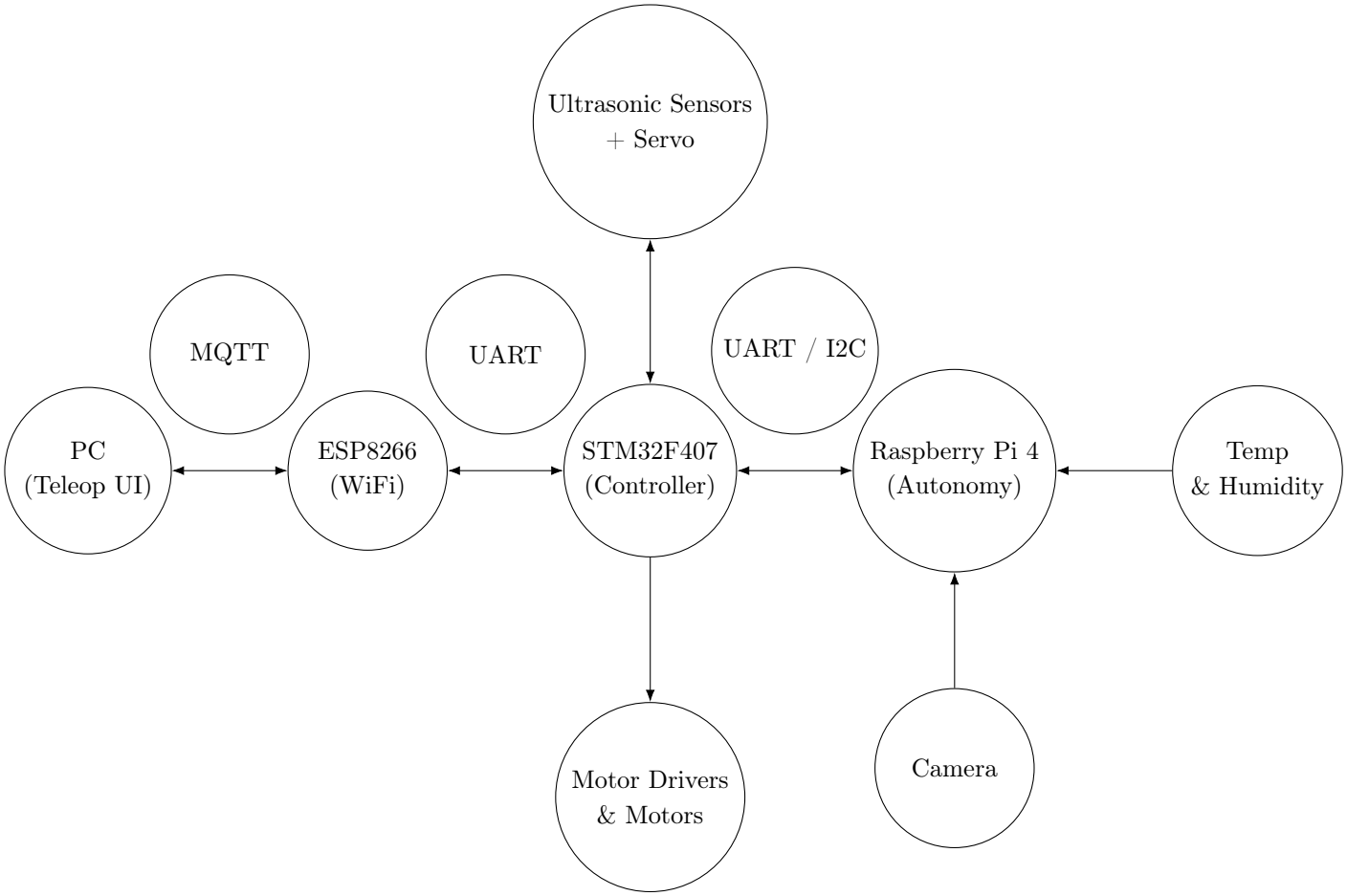


Figure 1: High-Level System Architecture Diagram

4.2 Communication Interfaces

- Mosquitto manages message routing between robot, dashboard, and control systems.
- Node-RED subscribes to telemetry and publishes control commands.
- MQTT secured via username/password.

5 Hardware Components and Specifications

Component	Description
STM32F407VGT6	Real-time control, FreeRTOS-based MCU
ESP32 DevKit	WiFi communication module with MQTT link
Raspberry Pi 4 (4GB)	Central node for AI, MQTT and data processing
Mosquitto Broker	Publishes/subscribes telemetry between modules
Node-RED	Dashboard UI for control and monitoring
Motor Drivers	Dual L298N, 10A capable
Motors (x4)	Encoder-based 12V DC motors
Ultrasonic Sensors	3 fixed + 1 servo mounted
Camera	Raspberry Pi V2 Module
DHT22	Temperature/Humidity sensing
IMU MPU9250	Motion tracking (optional)
Display	OLED/LCD showing system status
Battery	Li-ion 4S with protection and BMS

6 Software Architecture and Data Acquisition

6.1 Embedded Firmware (STM32F407)

- FreeRTOS handles motion, telemetry, and safety concurrently.
- UART DMA communication to Raspberry Pi.
- ISR-based safety trigger for instant stop.

6.2 Autonomous Intelligence (Raspberry Pi 4)

- Python and Node-RED workflows for real-time analytics.
- MQTT publisher/subscriber to Mosquitto broker.
- AI inference on captured frames to estimate collision risks.
- Continuous diagnostic monitoring.

6.3 Data Acquisition Using Mosquitto and Node-RED

- The Raspberry Pi publishes sensor values and AI diagnostics via MQTT.
- Node-RED dashboard visualizes them (gauges, charts, logs).
- Data are exportable to CSV or InfluxDB for post-analysis.
- Control commands (start, stop, emergency) can be issued remotely.
- Enables modular IoT connectivity for testing and challenge evaluation.